

CSA0619-Design and Analysis of Algorithm

28. Brute Force String Matching Algorithm:

main.py	Run	Output
<pre>1 # Brute Force String Matching 2 text = input("Enter the text: ") 3 pattern = input("Enter the pattern: ") 4 n = len(text) 5 m = len(pattern) 6 found = False 7 for i in range(n - m + 1): 8 match = True 9 for j in range(m): 10 if text[i + j] != pattern[j]: 11 match = False 12 break 13 if match: 14 print("Pattern found at index", i) 15 found = True 16 if not found: 17 print("Pattern not found")</pre>		<pre>Enter the text: LOGERRORFOUND Enter the pattern: ERROR Pattern found at index 3 === Code Execution Successful ===</pre>

Analysis:

The Brute Force String Matching algorithm compares the pattern with the text character by character. If a mismatch occurs, the pattern is shifted by one position and compared again. It is simple but performs many repeated comparisons, making it slow for large log files.

Optimization:

The Brute Force algorithm performs many repeated comparisons when searching large log files. As the size of the text increases, the search becomes slower.

Algorithms such as **KMP**, **Boyer–Moore**, and **Rabin–Karp** reduce unnecessary comparisons and provide faster searching.

Report Analysis

Case	Time Complexity
Best	$O(m)$
Average	$O(n \times m)$
Worst	$O(n^2)$

Space Complexity: $O(1)$

Where:

- **n = Length of the text**
- **m = Length of the pattern**

